

الگوریتم‌های موتور تطبیقی

از ریاضی Elo تا FSRS – استخراج کامل، مثال‌های عددی و
دستگیره‌های تنظیم

این کتابچه هر شش موتور الگوریتمی آزمونک را از صفر توضیح می‌دهد: انگیزه و مسئله، استخراج ریاضی گام‌به‌گام، مثال عددی با اعداد واقعی سند، مقایسه کامل با خانواده‌های جایگزین (IRT، Glicko-2، TrueSkill، BKT، DKT، لایتنر، Pimsleur، FSRS و Half-Life Regression) و در پایان هر فصل، فهرست دستگیره‌هایی که می‌توان برای تنظیم یا ویرایش هر الگوریتم چرخاند.

پدیدآور

پویا – جشنواره نوجوان خوارزمی، رشته رایانه / نرم‌افزار / هوش مصنوعی

مهندس همراه

سند برنامه توسعه نسخه ۱/۳ – مهر ۱۴۰۵

چیدمان فصل‌ها

نُه فصل اصلی + چهار ضمیمه – ترتیب، همان ترتیب «چرخه یک پاسخ» در سیستم است.

۱	نقشه کلی – چرخه یک پاسخ	چرا شش موتور جدا؟
۲	موتور Elo – سنجش توان	استخراج + Glicko-2، TrueSkill، IRT
۳	جایابی – مشکل شروع سرد	جلسه چهار سؤالی
۴	انتخاب سؤال – سه لایه فیلتر	ناحیه هدف ۶۵–۷۵٪
۵	نردبان سطح دسته‌ای (D16)	BKT، DKT، KST +
۶	SM-2 – مرور فاصله‌دار	+ لایتنر، FSRS، HLR، Pimsleur
۷	پیش‌بینی نمره	نمره وزن‌دار + رگرسیون $\lambda=0.85$
۸	چارچوب اجرا و قواعد پشتیبان	جلسه، اعتماد، تراکنش، snapshot
۹	جمع‌بندی تصمیم‌ها – چرا این‌ها؟	جدول انتخاب‌ها + نقشه دستگیره‌ها
الف	ضمیمه الف – جدول کامل پارامترها	مقدار، منبع، اثر تغییر
ب	ضمیمه ب – ده پرسش دآوری	پرسش + پاسخ کوتاه
ج	ضمیمه ج – واژه‌نامه اصطلاحات	انگلیسی ← فارسی
د	ضمیمه د – مراجع علمی	مقالات اصلی

هدف این سند «مالکیت الگوریتم‌ها» است: بعد از خواندن هر فصل باید بتوانی بگویی هر پارامتر چه می‌کند، چرا این مقدار انتخاب شده و اگر تغییری دهی چه اتفاقی می‌افتد. همه مقادیر پیش‌فرض با نشان «قابل و تو» در سند برنامه هم ثبت‌اند.

نقشه کلی – چرخه یک پاسخ

آزمونک از شش موتور الگوریتمی ساخته شده که هرکدام فقط یک سؤال را جواب می‌دهند: **جایابی** می‌گوید «اولین بار از کجا شروع کنم؟»، **Elo** می‌گوید «الان چقدر بلدم؟»، **انتخاب سؤال** می‌گوید «الان کدام سؤال را بدهم؟»، **نردبان سطح** می‌گوید «کجای مسیر هر دسته هستم و چه بخوانم؟»، **SM-2** می‌گوید «چه زمانی یادم می‌رود و باید مرور کنم؟» و **پیش‌بینی** می‌گوید «با این روند، فردا کجا می‌رسم؟». جداسازی این شش وظیفه عمدی است: هیچ الگوریتمی نباید همزمان دو نگرانی مختلف را مدیریت کند، چون آنگاه نه می‌توان جداگانه تستش کرد، نه جداگانه تنظیمش کرد، نه به داور توضیحش داد.

قاعده معماری سند برنامه صریح است: **«هیچ منطق الگوریتمی خارج از core نوشته نمی‌شود»**. یعنی تمام ریاضی این شش موتور در یک پکیج پایتون خالص به نام *core* زندگی می‌کند – بدون دسترسی به دیتابیس، بدون HTTP، بدون فریم‌ورک. بک‌اند FastAPI فقط *core* را صدا می‌زند و نتیجه را به وب می‌دهد. این تفکیک سه پیامد عملی دارد: نخست، هر الگوریتم را می‌توان با «تست طلایی» (اعداد دقیق سند) تست کرد بدون اینکه دیتابیس بالا بیاید؛ دوم، اگر خواستی یک الگوریتم را ویرایش کنی، دقیقاً می‌دانی کدام فایل و کدام تابع را باز کنی؛ سوم، برای داوری می‌توانی ادعای «تفکیک الگوریتم از رابط» را با کد اثبات کنی – ادعایی که داوران جشنواره به آن امتیاز می‌دهند.

۱/۱ چرخه کامل: از «سؤال بعدی» تا «پیش‌بینی فردا»

وقتی دانش‌آموزی یک گزینه را انتخاب می‌کند، در سرور یک زنجیره رویداد رخ می‌دهد که همه شش موتور را دست‌کم یک‌بار لمس می‌کند. این زنجیره را باید مثل یک کارخانه خط مونتاژ دید: ورودی خام «پاسخ + اعتماد + زمان» است و خروجی نهایی «سؤال بعدی + پروفایل به‌روز + صف مرور فردا».

(۱) پاسخ ثبت می‌شود با سه سیگنال: گزینه انتخابی، دکمه اعتماد («مطمئن بودم / حدس می‌زدم» – D14) و زمان پاسخ.

(۲) **Elo** اجرا می‌شود: توان سراسری دانش‌آموز و سختی همان سؤال هر دو آپدیت می‌شوند (فصل ۲).

(۳) **نردبان سطح** اجرا می‌شود: استریک غلط/درست دسته همان سؤال به‌روز می‌شود؛ اگر شرایط افت یا ارتقا فراهم باشد سطح تغییر می‌کند و اگر ضعف پایدار باشد بنر «مطالعه کن» فعال می‌شود (فصل ۵).

(۴) **SM-2** اجرا می‌شود: از ترکیب درستی، اعتماد و زمان، کیفیت q ساخته می‌شود و فاصله مرور بعدی همان مفهوم تعیین می‌شود (فصل ۶).

(۵) همه این‌ها در **یک تراکنش اتمیک** نوشته می‌شود، همراه با snapshot پارامترها – یا همه می‌نویسند یا هیچ‌کدام (فصل ۸).

(۶) برای سؤال بعدی، **سیاست انتخاب** با نوار سختی سطح جاری دسته، سؤال جدیدی برمی‌دارد (فصل ۴). پایان جلسه، **پیش‌بینی** با جلسه جدید به‌روز می‌شود (فصل ۷).

دو اصل طراحی هم روی کل چرخه حاکم‌اند. اول، **توضیح‌پذیری برای انسان**: هر تصمیم سیستم باید در یک جمله برای دانش‌آموز یا داور قابل بازگویی باشد – به همین دلیل پیش‌بینی زیر هشت جلسه نمایش داده نمی‌شود و افت سطح دسته با پیام مشخص «برای دسته X مطالعه کن» همراه است. دوم، **تکرارپذیری علمی**: هر رکورد پاسخ، مقدار لحظه‌ای همه پارامترها را در فیلد `params_snapshot` نگه می‌دارد تا تحلیل‌گر بتواند هر پاسخ را با همان پارامترهای لحظه پاسخ بازمحاسبه کند – این دقیقاً چیزی است که داور با «قابل تکرار بودن آزمایش» می‌سنجد.

فصل ۲

موتور Elo – سنجش توان

Elo در دهه ۱۹۶۰ توسط آرپد آلو، فیزیک‌دان و استاد شطرنج آمریکایی، برای ریتینگ بازیکنان شطرنج طراحی شد و بعدها توسط FIDE استاندارد شد. ایده مرکزی ساده اما عمیق است: **هم بازیکن عدد دارد، هم حریف** – و نتیجه هر برخورد، مقایسه‌ای بین دو عدد است که هر دو را جابه‌جا می‌کند. در آزمونک، برخورد «دانش‌آموز و سؤال» جای برخورد «بازیکن و بازیکن» می‌نشیند: توان دانش‌آموز R_s و سختی سؤال R_q هر دو روی یک مقیاس مشترک (حدود ۱۱۰۰ تا ۱۴۰۰) تعریف می‌شوند.

چرا این تفاوت بنیادی با نمره مدرسه است؟ نمره خام به سؤال‌های بسته شده بستگی دارد: ۷۰٪ در آزمون سخت و ۷۰٪ در آزمون آسان یک عدد یکسان‌اند، در حالی که دو توان کاملاً متفاوت را نشان می‌دهند. اما در Elo، اگر سؤال‌ها سخت‌تر باشند توان بالاتری می‌گیری – چون خود سؤال‌ها هم ریتینگ دارند و سیستم «شجاعت انتخاب آزمون سخت» را می‌بیند. این ویژگی، مقایسه بین دانش‌آموزان و بین زمان‌های مختلف را عادلانه می‌کند و همین است که به پیش‌بینی نمره (فصل ۷) یک ورودی پایدار می‌دهد.

۲/۱ استخراج گام به گام فرمول انتظار

ریشه ریاضی Elo مدل **Bradley-Terry** است: برای هر بازیکن i یک «قدرت مثبت» π_i تعریف می‌کنیم و احتمال برد A روی B را نسبت قدرت‌ها می‌گیریم:

$$P(A \text{ beats } B) = \frac{\pi_A}{\pi_A + \pi_B}$$

اکنون انتخاب نبوغ‌آمیز Elo: قدرت را نمایی از ریتینگ تعریف می‌کنیم، $\pi = 10^{R/400}$. با جای‌گذاری در کسر بالا، $10^{R_A/400}$ از صورت و مخرج فاکتور می‌شود:

$$P = \frac{10^{R_s/400}}{10^{R_s/400} + 10^{R_q/400}} = \frac{1}{1 + 10^{(R_q - R_s)/400}}$$

فرمول انتظار — قلب موتور (2.1)

این همان تابع لجستیک پایه ۱۰ است. دو نکته از این استخراج مهم است. اول، **معنای مقیاس ۴۰۰**: اختلاف ۴۰۰ امتیازی یعنی نسبت قدرت $10^{400/400} = 10$ برابر، یعنی شانس برد حدود ۹۱٪. اختلاف ۱۵۰ امتیازی (فاصله دو رده سختی آزمونک: ۱۲۰۰ مقابل ۱۳۵۰) شانس $10^{150/400} \approx 2.37$ برابر می‌دهد — یعنی حدود ۳۰٪ برای ضعیف‌تر. دقیقاً همان عددی که در مثال طلایی سند آمده.

دوم، این تابع **سیگموئیدی** است: در تفاضل صفر شانس ۵۰٪ است، در دنباله‌ها به ۰ یا ۱ اشباع می‌شود اما هرگز به آن نمی‌رسد. چرا اشباع خوب است؟ چون سیستم هرگز «مطمئن مطلق» نمی‌شود: حتی دانش‌آموز ۱۴۰۰ مقابل سؤال ۱۱۰۰ هم ۹٪ احتمال خطا دارد، پس یک غلط ناگهانی توانش را نمی‌سوزاند — فقط کمی پایین می‌آورد.

۲/۲ آپدیت دوطرفه و معنای واقعی K

بعد از هر پاسخ، هر دو ریتینگ حرکت می‌کنند. اگر S نتیجه واقعی باشد (۱ برای درست، ۰ برای غلط)، آپدیت به این شکل است:

$$R_s' = R_s + K_s \cdot (S - E) \quad ; \quad R_q' = R_q + K_q \cdot (E - S)$$

«در» سورپرایز K آپدیت دوطرفه — حاصل ضرب (2.2)

نکته کلیدی، عبارت $(S - E)$ است که به آن «سورپرایز» می‌گوییم: اگر نتیجه دقیقاً همان چیزی باشد که انتظار می‌رفت ($S = E$ در حد)، هیچ‌کس حرکت نمی‌کند؛ هرچه نتیجه دور از انتظار باشد، جابه‌جایی بزرگ‌تر است. ضریب K حداکثر سرعت حرکت را می‌سازد و از نگاه بهینه‌سازی ریاضی، آپدیت Elo یک **گام‌گردان** تصادفی است: سیستم دارد خطای پیش‌بینی‌اش را با گام‌های K کمینه می‌کند. K بزرگ = یادگیری سریع اما

ناپایدار (هر جلسه شانس‌آور، ریتینگ را می‌پراند)؛ K کوچک = پایدار اما کند. این نگاه، برنامه K را کاملاً توجیه می‌کند:

پارامتر	مقدار	چرا این مقدار؟
K_s - شروع	۳۲	در ابتدا توان واقعی ناشناخته است؛ حرکت سریع به سطح درست، هزینه‌اش پذیرفتنی است.
K_s - پس از ۵۰ پاسخ	۲۴	تخمین حالا تقریباً رسیده؛ حرکت آرام‌تر، نویز جلسه‌های شانس‌آور را فیلتر می‌کند.
K_s - پس از ۱۵۰ پاسخ	۱۶	فاز پایداری: تغییرات توان باید واقعی و تدریجی باشد، نه بازتاب نوسان روزانه.
K_q - سؤال	$8 \leftarrow 4$ پس از ۴۰ پاسخ	سختی سؤال از تجمیع ده‌ها دانش‌آموز می‌آید؛ هر پاسخ واحد اطلاعات کمتری درباره سؤال دارد تا درباره یک دانش‌آموز. پس K کوچک‌تر و کاهشی.

۲/۳ مثال عددی کامل (تست طلایی سند)

دانش‌آموز توان ۱۲۰۰، سؤال سختی ۱۳۵۰ - پاسخ درست

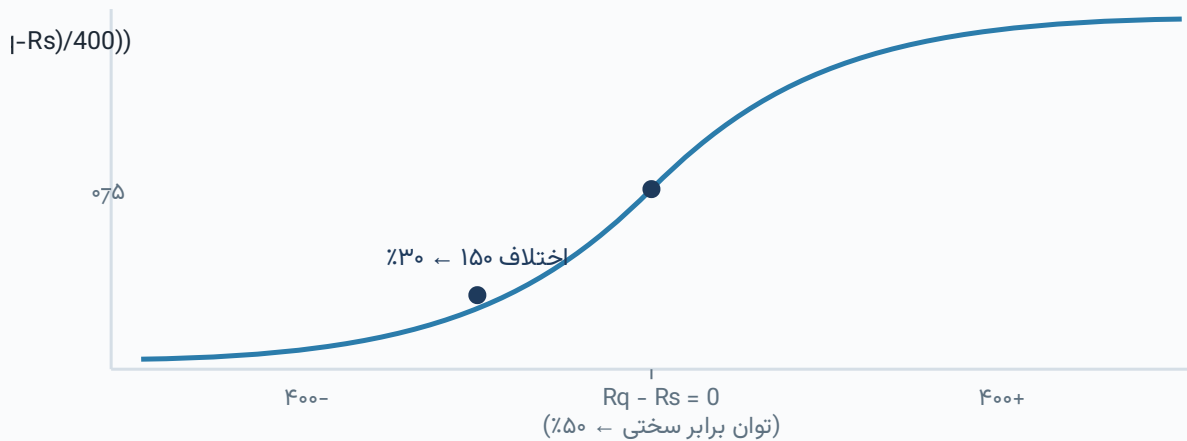
گام ۱ - انتظار: تفاضل $R_q - R_s = 150$ ؛ $10^{150/400} = 10^{0.375} \approx 2.371$ ؛ پس $E = 1/(1+2.371) \approx 0.296$ - شانس درست‌زدن فقط ۲۹/۶٪.

گام ۲ - سورپرایز: پاسخ درست بود، پس $S - E = 1 - 0.296 = 0.704$.

گام ۳ - آپدیت دانش‌آموز: $R_s' = 1200 + 32 \times 0.704 = 1222.5$ - حدود ۲۲ امتیاز جهش، چون کاری بهتر از انتظار انجام شد.

گام ۴ - آپدیت سؤال: $R_q' = 1350 + 8 \times (0.296 - 1) = 1350 - 5.6 = 1344.4$ - سؤال کمی آسان‌تر شد؛ سؤالی که همه به آن جواب درست بدهند به تدریج به سختی واقعی‌اش می‌رسد. این همان «رأی جامعه» بر سختی است.

تست طلایی: مقدارهای ۱۲۲۲/۵ و ۱۳۴۴/۴ با خطای کمتر از ۰/۱ باید در تست core بازتولید شوند.



شکل ۲ - منحنی لجستیک انتظار: در تفاضل صفر، ۵۰٪؛ هر ۴۰۰ امتیاز، شانس را ده برابر می‌کند

۲/۴ خانواده رقیب – Glicko-2، IRT و TrueSkill

IRT (نظریه پاسخ به سؤال) جامع‌ترین پاسخ علمی به همین مسئله است و از دهه ۱۹۶۰ در آزمون‌های استاندارد بین‌المللی (توفل، PISA، GRE) استفاده می‌شود. مدل سه‌پارامتری آن احتمال درستی را این‌طور می‌سازد:

$$P(\text{correct} | \theta) = c + (1 - c) \cdot \frac{\exp[a(\theta - b)]}{1 + \exp[a(\theta - b)]}$$

شانس حدس c ، قدرت تمایز a ، سختی b ، توان θ — سه پارامتری IRT (2.3)

سه پارامتر هر سؤال معنا دارد: b سختی (معادل R_q)، a تمایز (چقدر این سؤال توانِ باطل‌کننده است — شیب منحنی) و c سقف حدس (برای چهارگزینه‌ای حدود ۰/۲۵). از دید ساختاری، Elo یک IRT دوپارامتری با a ثابت و $c = 0$ است؛ برتری IRT در همان دو پارامتر اضافه است، اما بهای آن برآورد پسروری (Bayesian/MLE) روی کل داده است — یعنی هر بار داده جدید می‌آید، بازبرآورد دسته‌جمعی لازم دارد، در حالی که Elo با یک جمع و تفریق ساده، لحظه‌ای به‌روز می‌شود و روی موبایل هم جا می‌شود.

Glicko-2 (مارک گلیکمن، دانشگاه بوستون) نسخه پیشرفته Elo است که به هر ریتینگ یک **انحراف معیار RD** اضافه می‌کند — «چقدر از تخمین مطمئنیم» — و یک پارامتر وولاتیلتی برای تغییرپذیری عملکرد. دو بازیکن بی‌فعال، RDشان خودبه‌خود بزرگ می‌شود؛ یعنی سیستم اذعان می‌کند تخمینش کهنه شده است. این دقیقاً همان «شاخص عدم قطعیت» است که خروجی جایابی آزمونک (فصل ۳) لازم دارد و می‌توان در نسخه‌های بعد به Elo اضافه‌اش کرد — ولی پیاده‌سازی کاملش شش تا هفت برابر Elo کد می‌برد و تفسیرش برای دبیر و دانش‌آموز سخت‌تر است.

TrueSkill (مایکروسافت ریسرچ، برای Xbox Live) مدل گاوسی کامل است: توان هر بازیکن یک توزیع نرمال $N(\mu, \sigma^2)$ است و آپدیت با گراف‌های فاکتور بیزی انجام می‌شود؛ رنک نمایشی $\mu - 3\sigma$ است تا بچه‌های کم‌تجربه بالای رتبه نگیرند. TrueSkill برای بازی تیمی چندنفره بی‌رقیب است، اما برای «یک دانش‌آموز در برابر یک سؤال» پیچیدگی‌اش اسراف منابع است. جدول زیر جمع‌بندی انتخاب آزمونک است:

معيار	Elo (انتخاب ما)	IRT 3PL	Glicko-2	TrueSkill
هزینه محاسبه در هر پاسخ	یک جمع و تفریق – لحظه‌ای	بازبرآورد دسته‌جمعی دوره‌ای	میانہ – چند فرمول بسته	تقریب بیزی تکراری
حجم داده لازم	کم – از اولین پاسخ	زیاد (دهها پاسخ به ازای هر سؤال)	کم	متوسط
توضیح‌پذیری برای داور/دبیر	بالا – دو عدد و یک فرمول	متوسط – سه پارامتر به‌ازای هر سؤال	متوسط – RD باید توضیح داده شود	پایین – گراف فاکتور
مدل‌سازی حدس	غیرمستقیم (دکمه اعتماد D14)	مستقیم با پارامتر C	غیرمستقیم	غیرمستقیم
عدم قطعیت داخلی	ندارد (K برنامه‌ای جایگزین)	دارد	دارد (RD)	دارد (σ)

جدول ۱-۲ – مقایسه خانواده سنجش توان. انتخاب آزمونک: Elo برای موتور اصلی؛ پارامتر C مدل‌سازی حدس را با دکمه اعتماد (سیگنال انسانی، فصل ۸) جبران می‌کنیم؛ RD هم در فاز جایابی با شاخص ساده پراکندگی تقریب می‌شود.

دستگیره‌های قابل تنظیم این فصل

K_q / decay

۸ ← ۴ پس از ۴۰ پاسخ

کاهشش یعنی سختی سؤال‌ها سفت‌تر و مقاوم‌تر می‌شود؛ افزایشش برای فاز پرکردن بانک سؤال جدید منطقی است (جذب سریع رأی جامعه).

K_s schedule

۳۲ ← ۲۴ (۵۰ پاسخ) ← ۱۶ (۱۵۰ پاسخ)

بالا بردن گام‌ها = واکنش سریع‌تر به تغییر واقعی توان ولی ناپایداری بیشتر در برابر شانس. کاهش آستانه‌ها (مثلاً ۲۵/۱۰۰) یعنی «سریع‌تر به حالت پایدار برو».

Initial R_s

جایابی (فصل ۳) – نه مقدار ثابت

اگر روزی خواستی جایابی را کوتاه کنی، مقدار پیش‌فرض ۱۲۰۰ به‌عنوان محافظ fallback در core هست؛ ولی دقت جایابی از آن بهتر است.

Scale constant

۴۰۰ در نمای ریتینگ

عدد بزرگ‌تر = منحنی لجستیک کش‌آمده و اثر تفاضل‌ها ملایم‌تر؛ تغییرش همه ثابت‌های تاریخی (۱۱۰۰-۱۴۰۰) را هم باید جابه‌جا کند. توصیه: دست نزن.

فصل ۳

جایابی – مشکل شروع سرد

هر موتور تطبیقی یک ماده منفجره کوچک دارد: دانش‌آموز جدید هنوز هیچ پاسخی ثبت نکرده، پس Elo هیچ داده‌ای برای تخمین توانش ندارد. به این «شروع سرد» (cold start) می‌گویند و راه‌حل‌های بد آن می‌تواند کل اعتبار سیستم را در پنج دقیقه اول بسوزاند: اگر به دانش‌آموز قوی، سؤال‌های آسان بدهی،

حوصله‌اش سر می‌رود؛ اگر به دانش‌آموز ضعیف سؤال‌های سخت بدهی، ناامید می‌شود و اپ را می‌بندد. بدترین حالت، هر دو اتفاق در اولین تجربه محصول است – جایی که هیچ فرصت دومی وجود ندارد.

گزینه‌های موجود در بازبینی طراحی (گپ G1 سند) سه تا بود: **خوداظهاری** («خودت را چه امتیازی می‌دهی؟») که ارزان است ولی تقریباً بی‌ارزش – نوجوان‌ها یا متواضع یا جسورند و هیچ‌کدام دقیق؛ **پیش‌فرض ثابت ۱۲۰۰** که ساده است ولی به هر چهار نفر یک نقطه شروع یکسان می‌دهد و جلسه‌های اول هر چهار نفر را خراب می‌کند؛ و **نمره پیش‌آزمون** که دقت دارد ولی دو مشکل دارد: همه‌ی کاربران پیش‌آزمون ندارند (فقط شرکت‌کنندگان آزمایش علمی دارند) و به‌علاوه سند صریح گفته در سناریوی آزمایش، نمره پیش‌آزمون فقط برای «بلوکه‌سازی تصادفی» استفاده می‌شود نه توان اولیه – تا پیش‌آزمون و موتور مستقل بمانند و منبع نشت داده نشود.

انتخاب مصوب (D1) **جلسه جایابی** است: خودِ موتور، سطح را پیدا کند. جزئیات طراحی:

- **۴ سؤال با سختی پراکنده** از استخر ۱۱۵۰ / ۱۲۵۰ / ۱۳۵۰. «پراکنده» کلیدی است – دنباله صعودی یا نزولی به دانش‌آموز اجازه می‌دهد الگو بفهمد و حدس سیستماتیک بزند؛ پراکندگی این را می‌بندد. محدوده مجاز سند ۳ تا ۵ سؤال است و پیش‌فرض مهندسی ۴.
- **K ثابت روی ۳۲** در کل جایابی: منطق این است که در جایابی، توان ناشناخته است و باید سریع‌ترین همگرایی اتفاق بیفتد؛ برنامه کاهشی K هنوز فعال نمی‌شود.
- **تشخیص کیفیت جایابی**: اگر پراکندگی پاسخ‌ها بالا باشد – مثلاً سخت را درست و آسان را غلط بزند – سیگنال ناسازگار است؛ جلسه با ۲ سؤال تکمیلی ادامه می‌یابد تا تخمین جا بیفتد. این همان «شاخص عدم قطعیت» سند است: جایابی بد = عدم قطعیت بالا = سؤال بیشتر.
- **خروجی: R_s** اولیه ذخیره می‌شود و جلسه عادی ۱۰ سؤالی آغاز می‌شود. تجربه کاربری هم صادقانه طراحی شده: پیام «چند سؤال برای شناخت سطحت» – نه ادعای آزمون سرنوشت‌ساز.

جایابی در عمل – دو سناریو

سناریوی سالم: پاسخ‌ها «آسان ✓، متوسط X، سخت X، متوسط ✓» – الگو با توانِ حوالی ۱۲۰۰ سازگار است. توان اولیه حدود ۱۲۱۰-۱۲۳۰ در می‌آید و جلسه عادی شروع می‌شود.

سناریوی ناسازگار: «آسان X، متوسط ✓، سخت ✓، آسان ✓» – دو ناسازگاری مستقیم. شاخص پراکندگی بالا می‌رود؛ دو سؤال تکمیلی (بازه ۱۱۵۰-۱۳۵۰) سرو می‌شود و تخمین با ۶ نقطه ادامه می‌یابد. اگر باز هم ناسازگار ماند، سیستم با K بزرگ‌تر به کار می‌رود – یعنی توانش زودتر خودش را نشان می‌دهد.

یک نکته ریز اما مهم برای دفاع: جایابی «تست» نیست، «تنظیم خودکار» است. هیچ نمره‌ای نمایش داده نمی‌شود، قبولی و مردودی در کار نیست و اگر نوجوان بد کند، سیستم به‌جای تنبیه، فقط شناختش را کندتر و محتاطانه‌تر ادامه می‌دهد. این تمایز دقیقاً همان چیزی است که حس «دستیار» را در برابر حس «امتحان‌دهنده» می‌سازد – و موضع رقابتی آزمونک با اپ‌های آزمون‌ساز سنتی است.

extra_on_bad_fit

۲ سؤال تکمیلی

سقف حفاظتی است تا حلقه بدشکلی نشود؛ اگر نرخ «جایابی ناموفق» در داده واقعی بالای ۲۰٪ آمد، آستانه تشخیص پراکندگی را بازبینی کن، نه این عدد را.

placement_length

۴ سؤال (محدوده ۳-۵)

کوتاه‌تر = تجربه کاربری بهتر ولی تخمین خشن‌تر. اگر داده آزمایش میدانی نشان دهد پراکندگی اولیه زیاد است، ۵ را امتحان کن.

difficulty_pool

۱۳۵۰ / ۱۲۵۰ / ۱۱۵۰

فاصله ۱۰۰ امتیازی بین رده‌ها یعنی نسبت شانس تقریباً ۱/۸× بین هر دو رده متوالی - به قدری که پراکندگی واقعاً «پراکندگی» باشد.

فصل ۴

انتخاب سؤال - سه لایه فیلتر

موتور Elo می‌گوید «الان چقدر بلام»، اما تصمیم بعدی این است: «خب، پس الان کدام سؤال را سرو کنم؟». این تصمیم از نظر آموزشی مهم‌ترین لنگر کل سیستم است، چون ناحیه‌ای که سؤال‌ها انتخاب می‌شوند، تعیین می‌کند دانش‌آموز در «منطقه تقریبی توسعه» بماند یا نه - ناحیه‌ای که در آن کار سخت است اما نه به قدری که ناامیدکننده باشد. سیاست آزمونک سه لایه دارد که از کلان به خرد اجرا می‌شوند.

۴/۱ لایه اول - ناحیه هدف ۶۵٪ تا ۷۵٪

فرمول انتظار (۲/۱) برای هر سؤالِ کاندید یک عدد می‌دهد: شانس موفقیت این دانش‌آموز روی این سؤال. سیاست سند: سؤالی انتخاب کن که این شانس در بازه ۶۵٪ تا ۷۵٪ باشد؛ عملیاتی‌اش، سؤالی با کمینه فاصله از ۷۰٪ (میان‌ه ناحیه):

$$q^* = \operatorname{argmin}_{q \in \text{Candidates}} |E(R_s, R_q) - 0.70|$$

معیار کمینه فاصله از نقطه هدف ۷۰٪ - از اسلایدهای سند (4.1)

چرا ۷۰٪ و نه ۵۰٪؟ در ۵۰٪، دانش‌آموز عملاً با شانس سکه پیش می‌رود - اضطراب بالا و بازخورد آموزشی کم (هر دو مسیر درست و غلط به یک اندازه شانس دارند و «چرا درست شد» روشن نمی‌شود). در ۹۰٪، تمرین تکراری بدون چالش است. ناحیه ۶۵-۷۵ تعادل سند است: به قدری موفقیت که انگیزه بماند، به قدری شکست که یادگیری رخ دهد. نکته مهم: این ناحیه قابل تنظیم است و در آزمایش علمی حساسیت‌سنجی می‌شود - یعنی می‌توانی با داده واقعی بگویی «ناحیه X بهترین رشد را داد» و همان را دفاع کنی؛ این یکی از زیباترین نتایج قابل ارائه به داوری است.

۴/۲ لایه دوم – امتیاز ارزش آموزشی

معمولاً چند سؤال به طور همزمان داخل ناحیه هدف اند؛ در این جا سیاست از «سخت کدام» به «مفید کدام» سوییچ می کند. هر کاندید یک امتیاز ارزش آموزشی می گیرد که جمع وزنی سه مؤلفه است:

$$V(q) = w_1 \cdot Weakness(cat(q)) + w_2 \cdot Importance(topic(q)) + w_3 \cdot Recency(q)$$

امتیاز ارزش آموزشی — وزن ها پیش فرض مهندسی و قابل وتو (4.2)

- $Weakness(cat)$ — وزن ضعف دسته از نردبان سطح (فصل ۵): دسته ای که سطحش پایین آمد یا نرخ موفقیت اخیرش کم است، امتیاز بالا می گیرد. این حلقه اتصال D16 به انتخاب سؤال است: سیستم نه فقط «سخت یا آسان» بلکه «کجا ضعف داری» را هدایت می کند.
- $Importance(topic)$ — اهمیت مبحث در فصل (پیش نیاز بودن، وزن در امتحان نهایی)؛ یک جدول ساده در محتوا نگه داشته می شود و نویسندگان محتوا آن را پر می کنند.
- $Recency(q)$ — فاصله زمانی از آخرین برخورد با آن مبحث؛ مباحث رها شده شده کم کم بالا می آیند تا «فراموشی محض» رخ ندهد.

۴/۳ لایه سوم – محدودیت مواجهه

آخرین فیلتر حفاظتی است: سؤالی که در ۲۰ پاسخ اخیر دیده شده از کاندیدها حذف می شود ($exposure_window = 20$ — گپ G8 سند). بدون این فیلتر، سیاست ارزش آموزشی می تواند عاشق یک سؤال شود و آن را در هر جلسه تکرار کند؛ نتیجه، «حفظ سؤال» و تورم مصنوعی نمره است. عدد ۲۰ هم خودش دستگیره است: پنجره کوتاه تر = تنوع بیشتر ولی مصرف سریع تر بانک سؤال؛ با بانک ۱۵۰ سؤالی فعلی، ۲۰ تعادل معقولی است.

رقیب این فصل: در IRT استاندارد، انتخاب سؤال با «ماکسیم اطلاعات» انجام می شود — سؤالی که واریانس تخمین توان را بیشترین کاهش می دهد (تقریباً همان سؤالی که شانس موفقیت رویش نزدیک ۵۰٪ است). این سیاست از نظر آماری بهینه است اما دو ضعف برای محصول ما دارد: ناحیه هدفش به ۵۰٪ میل می کند (در مقایسه با ۷۰٪ سند — یعنی تجربه سخت تر و پرتنش تر) و هیچ جای تعبیه «ضعف دسته» و «اهمیت مبحث» ندارد، در حالی که این دو دقیقاً هوش آموزشی آزمونک اند. رویکرد آزمونک، به جای بهینگی خالص آماری، بهینگی آموزشی را با فیلتر آماری ناحیه هدف ترکیب می کند.

w1 / w2 / w3**وزن‌های ارزش آموزشی**

w1 بالا = سیستم مدافع ضعف‌ها؛ w3 بالا = سیستم یادآور مبحث‌های رهاشده. پیش‌فرض: w1 بزرگ‌ترین. تغییر این سه، شخصیت سیستم را عوض می‌کند.

target_zone

۰/۶۵ تا ۰/۷۵ (مرجع ۰/۷۰)

کشیدن ناحیه به پایین (۰/۶۵-۰/۵۵) = چالش سخت‌تر و فشار بیشتر؛ بالا (۰/۷۵-۰/۸۵) = اعتمادساز، مناسب دانش‌آموز ناامید. حساسیت‌سنجی در آزمایش میدانی انجام می‌شود.

exposure_window

۲۰ پاسخ

با رشد بانک سؤال به ۳۰۰+ می‌توانی ۱۵ کنی (تنوع بیشتر در بانک بزرگ)؛ با بانک کوچک MVP هشتادتایی، ۲۰ سقف امن است.

فصل ۵

نردبان سطح دسته‌ای (D16)

تا این‌جا سیستم یک عدد سراسری دارد: «این دانش‌آموز مجموعاً چقدر بلد است». اما «مجموعاً» جواب مناسبی برای دو سؤال حیاتی نیست: «کجای ریاضی‌ام ضعیف است؟» و «الان دقیقاً چه بخوانم؟». یک دانش‌آموز با توان ۱۲۵۰ می‌تواند در «قاعده زنجیره‌ای» عالی و در «حد از چپ» ضعیف باشد؛ اگر سیستم فقط عدد سراسری را ببیند، برای هر دو مبحث همان سختی را سرو می‌کند و در دسته ضعیف، تجربه ناامیدکننده می‌سازد.

نسخه نخست طراحی، پاسخ مفصل‌تری داشت: «موتور بدفهمی» – هر گزینه غلط هر سؤال به یک گره بدفهمی مشخص (مثل «مشتق را تفاضل می‌فهمد نه نرخ لحظه‌ای») مپ می‌شد و با قاعده دو شاهد، بدفهمی تثبیت‌شده اعلام می‌شد. در بازبینی طراحی، پویا این مکانیزم را وتو کرد (D16): نویسندگی مپ بدفهمی برای هر گزینه، بار تولید محتوا را دو تا سه برابر می‌کند و برای تیم فعلی غیرقابل انجام است. تصمیم نهایی: **فرمت حفظ، مکانیزم عوض**. ستون *distractor_map*، شیت *misconceptions* و سه جدول دیتابیس در اسکیمای بمانند (اختیاری، پیش‌فرض خالی) برای فعال‌سازی آینده؛ و مکانیزم فعال جایگزین شود با چیزی که پویا طراحی کرد: **نردبان سطح دسته‌ای**.

۵/۱ مدل داده: سه چیز به‌ازای هر دسته

هر سؤال دو مشخصه دارد: **دسته** (همان فیلد *topic* موجود – مثلاً «قاعده زنجیره‌ای») و **سطح ۱ تا ۳** (همان سه رده سختی موجود ← ۱۱۵۰ / ۱۲۵۰ / ۱۳۵۰). به همین دلیل این مکانیزم «تقریباً مجانی» بود: هیچ

ستون جدیدی لازم نشد. برای هر دانش‌آموز، به‌ازای هر (درس، دسته) یک ردیف پروفایل در جدول $ability_state$ نگه می‌داریم با این حالت:

$$State(user, cat) = \{ L \in \{1,2,3\}; s_w; s_r; w; H_5 \}$$

H_5 بافر پنج پاسخ آخر، w شمارنده ضعف، s_r استریک درست، s_w استریک غلط، L سطح جاری (5.1)

L تعیین می‌کند سؤال‌های این دسته از کدام نوار سختی سرو شوند؛ دو استریک «پیاپی بودن» را نگه می‌دارند؛ شمارنده ضعف w می‌شمارد چند بار دانش‌آموز در کف سطح ۱ سقوط کرده (سیگنال «بدتر از سطح آسان هم»); و بافر H_5 پنج نتیجه آخر همان دسته را برای محاسبه نرخ موفقیت نگه می‌دارد.

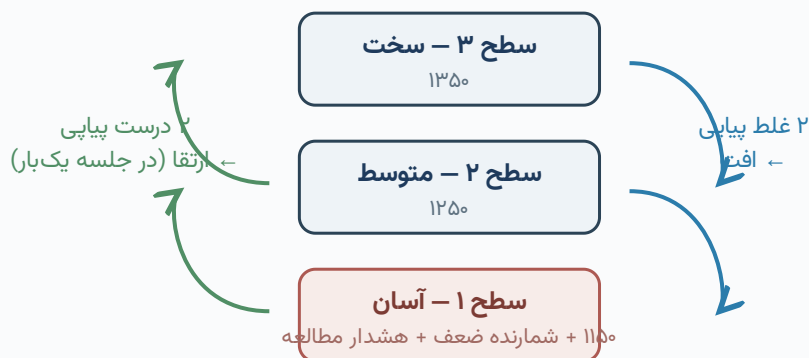
۵/۲ قواعد انتقال – کامل و بدون استثنا

رویداد	به‌روزرسانی	قید
پاسخ غلط در دسته c	$L = \max(1, : s_w \geq 2$ اگر $s_r = 0$ ؛ $s_w += 1$ ($L-1$) و ری‌ست استریک؛ اگر L قبل هم ۱ بود: $w += 1$	افت سطح همیشه با ری‌ست استریک همراه است تا حلقه بی‌نهایت افت‌های متوالی نگیرد
پاسخ درست در دسته c	$s_r += 1$ ؛ $s_w = 0$ اگر $s_r \geq 2$ و هنوز در این جلسه ارتقا نگرفته: $L = \min(3, L+1)$ و ری‌ست استریک	حداکثر یک ارتقا در هر جلسه برای هر دسته («یواش یواش»)
پایان هر پاسخ	نتیجه به H_5 اضافه می‌شود (پنجره لغزان ۵ تایی)	مبنای محاسبه نرخ موفقیت اخیر
بررسی هشدار	اگر $L = 1$ و نرخ درست H_5 کمتر از ۳۰٪: بنر «برای دسته X مطالعه کن» در داشبورد و خلاصه جلسه	هر دو شرط هم‌زمان لازم‌اند
بررسی دبیر	اگر سطح دسته c در ۵ جلسه آخر ۳ بار یا بیشتر تغییر کرده: به صف بازبینی دبیر می‌رود	هشدار نوسان – جانشین انسانی «قابل اعتراض بودن تشخیص»

جدول ۵-۱ – ماشین حالت نردبان. پارامترهای $n=2$ ، $m=2$ و آستانه ۳۰٪ پیش‌فرض مهندس‌اند و قابلیت وتو دارند (پویا هر وقت رأی بدهد جایگزین می‌شود).

سه طراحی دقیق در این قواعد ارزش دفاع دارند. **چرا استریک (پیاپی) و نه مجموع؟** دو غلط با وسطش یک درست، الگوی خطای پراکنده است – شاید بی‌حوصلگی، شاید غلط تایپ؛ اما دو غلط پیاپی در یک دسته، الگوی «اینجا چیزی را نمی‌فهمم» است. استریک نویز را فیلتر می‌کند بدون اینکه ریاضی پیچیده شود. **چرا حداکثر یک ارتقا در جلسه؟** چون یک جلسه شانس‌آور (یا جلسه‌ای که بچه از دوستش جواب گرفت) نباید دو سطح بپرد؛ ارتقا باید از جلسه به جلسه «رسوب» کند، مثل یادگیری واقعی. **چرا هشدار مطالعه دو شرط دارد؟** سطح ۱ تنها، برای دانش‌آموزی که تازه سقوط کرده طبیعی است؛ ۳۰٪ ضعیف است ولی اگر کل پنج

پاسخ اخیرش در سطح آسان هم کمتر از ۳۰٪ باشد، مشکل دیگر «سختی سؤال» نیست – مشکل «مفهوم خوانده نشده» است و راه حل الگوریتم نیست، مطالعه است. صادق ترین کاری که سیستم می کند همین است که بگوید: «من کاری نمی توانم بکنم؛ برو اول بخوان».



شکل ۵ – نردبان سه سطحی هر دسته: سقوط با دو غلط پیاپی، ارتقای محتاطانه با دو درست پیاپی

۵/۳ خانواده رقیب – BKT و DKT و KST

BKT (ردیابی دانش بیزی) – کلاسیک ترین مدل «وضعیت یادگیری هر مفهوم» (کوربت و اندرسون، ۱۹۹۴؛ پایه سیستم های تیوتر هوشمند از Carnegie Mellon). BKT برای هر مفهوم یک متغیر پنهان $P(L)$ نگه می دارد: «احتمال اینکه دانش آموز این مفهوم را واقعاً یاد گرفته باشد». چهار پارامتر دارد: احتمال اولیه P_0 ، نرخ یادگیری T ، شانس حدس g و شانس لغزش s . با هر مشاهده، قضیه بیزی وضعیت را به روز می کند – مثلاً برای پاسخ درست:

$$P(L|correct) = \frac{P(L)(1-s)}{P(L)(1-s) + (1-P(L))g} ; \text{ سپس } P^* = P(L|correct) + (1-P(L|correct)) \cdot T$$

پس از مشاهده پاسخ درست + قدم یادگیری BKT به روزرسانی (5.2)

BKT از نظر مفهومی خوشگل ترین پاسخ به سؤال «نمی دانم بچه واقعاً بلده یا حدس زده» است – گره g و s همین ها را مدل می کنند. اما برای آزمونک دو مانع عملی دارد: نخست، برآورد چهار پارامتر خوب به ازای هر مفهوم، به داده پایدار هر مفهوم نیاز دارد (ده ها پاسخ به ازای هر دسته برای هر دانش آموز یا برای کل جمعیت) – داده ۴۰ آزمودنی ما در ۱۰ هفته به سختی این را می دهد و بدترین حالت این است که پارامترها با داده کم، برآورد سرگردان شوند. دوم، خروجی BKT «احتمال» است – «۷۰٪ بلده» – که برای دانش آموز و دبیر قابل اقدام مستقیم نیست؛ نردبان همین را به «سطح ۲ از ۳، دو درست پیاپی تا ارتقا» ترجمه می کند که هر نوجوانی می فهمد.

DKT (ردیابی دانش عمیق) – نسل جدید همین ایده با شبکه بازگشتی (LSTM) است (پیچ و همکاران، ۲۰۱۵): دنباله کل پاسخ ها وارد شبکه می شود و در هر لحظه برداری از احتمال تسلط همه مفاهیم بیرون می دهد. DKT در مقاله اصلی روی داده صدها هزار دانش آموز آموزش دید و در مسابقات knowledge

tracing رکورد دارد؛ ولی سه ویژگی برای ما نامناسب است: نیاز به داده انبوه، خروجی جعبه‌سیاه غیرقابل توضیح (داور می‌پرسد «چرا سطحش را پایین آوردی؟» و جواب بردار ۱۰۰ بعدی است!) و وابستگی به زیرساخت GPU که بودجه پروژه دانش‌آموزی نیست. در روایت دفاعی، DKT را به‌عنوان «مسیر تحول آینده» نام می‌بریم – نشان می‌دهیم می‌دانیم از کجا می‌توان آمد – نه به‌عنوان گزینه فعلی.

KST (نظریه فضای دانش) – رویکرد صوری‌تر (دوین‌یون و فالماگن، ۱۹۸۵): مجموعه‌ای از «حالت‌های دانش» تعریف می‌شود (کدام مفاهیم با هم شناخته‌اند) و آزمون تطبیقی، سریع‌ترین مسیر به حالت درست را می‌جوید. ریاضیاتش زیباست ولی طراحی‌اش برای هر بانک سؤال، تحلیل وابستگی مفاهیم دستی سنگین می‌خواهد؛ همان‌جایی است که بار محتوایی دوباره از آستانه تیم ما رد می‌شود.

معیار	نردبان (انتخاب ما)	BKT	DKT	KST
داده لازم برای شروع	تقریباً صفر – قواعد ثابت	چهار پارامتر به‌ازای هر مفهوم	صدها هزار پاسخ	گراف مفهومی دستی
توضیح‌پذیری	کامل – سه قاعده آشکار	متوسط – احتمال خروجی	ضعیف – جعبه‌سیاه	متوسط
بار محتوایی	صفر اضافه – از topic و سختی موجود	کالیبراسیون پارامترها	پایپ‌لاین ML	تحلیل مفاهیم سنگین
خروجی قابل اقدام برای نوجوان	سطح + پیام مطالعه	احتمال تسلط	بردار احتمال‌ها	حالت دانش
شبیه‌سازی خطای ضعف پایدار	شمارنده ضعف در کف	فتادگی $P(L)$	فتادگی بردار	محدودیت حالت

جدول ۵-۲ – مقایسه خانواده «تشخیص وضعیت مفهومی». نردبان، BKT را می‌شود در فاز دو با داده انباشته‌شده فعال کرد – snapshot پارامترها برای همین نگه داشته می‌شود.

m (rise streak)

۲ درست پیایی

کاهشش ($m=1$) حس پیشرفت سریع‌تری می‌دهد ولی سطح‌ها را شکننده می‌کند؛ سقف «یک ارتقا در جلسه» را با m از هم جدا نگه دار.

n (drop streak)

۲ غلط پیایی

$n=1$ یعنی هر غلط، سطح را می‌اندازد - واکنشی ولی پرنوسان؛ $n=3$ محتاطانه ولی کند. ۲ تعادل سند است. اگر داده نشان دهد سطح‌ها پرش‌های ناخواسته دارند، اول n را بالا ببر.

oscillation_alert

۳ تغییر در ۵ جلسه ← صف دبیر

این شیر اطمینان انسانی است. اگر دبیر صف پرشدگی دید، یعنی یکی از n/m آستانه‌ها باید سخت‌گیرانه‌تر شود - عیب‌یابی از همین‌جا شروع کن.

study_alert

سطح ۱ + موفقیت $>30\%$ در ۵ پاسخ

$30\% =$ یک از سه. بالا بردنش (مثلاً 40%) بنرهای بیشتری می‌سازد - خطر «کری هشدار» و بی‌اثر شدن پیام برای دانش‌آموز.

فصل ۶

SM-2 – مرور فاصله‌دار

تمام موتورهای قبل به یک سؤال تازه می‌رسند: «بعد از چند روز، بچه چه چیزی را یادش می‌رود؟». منحنی فراموشی ابینگه‌اوس (۱۸۸۵) نشان داد حافظه به‌صورت نمایی افت می‌کند - و مهم‌تر، هر «نجات» درست قبل از فراموشی، منحنی را صلب‌تر می‌کند. **مرور فاصله‌دار** یعنی دقیقاً در همین لحظه‌ها تزریق تکرار: نه زودتر (اتلاف وقت روی چیزی که بلدی) و نه دیرتر (باید از صفر یاد بگیری). الگوریتم SM-2 توسط پیوتر وُژنیاک، دانشجوی لهستانی، در ۱۹۸۷ منتشر شد و تا امروز هسته نرم‌افزار SuperMemo و هزاران فرزند آن (از جمله Anki در نسخه‌های کلاسیک) است.

در آزمونک، مرور زیرساخت جدایی نیست - همان نوآوری ۳ سند: **مرور زیرساخت خود آزمون است**. سؤال‌هایی که موعد مرورشان رسیده، در جلسه عادی هم می‌توانند سرو شوند (اولویت در سیاست انتخاب، فصل ۴). این انتخاب تجربی مهمی است: «اپ مرور جدا» در واقعیت کم‌استفاده می‌شود، اما تزریق درون جلسه، مرور را به عادت جلسه تبدیل می‌کند.

۶/۱ کیفیت هر برخورد: q از سه سیگنال

SM-2 از هر برخورد یک عدد کیفیت q در بازه ۰ تا ۵ می‌سازد و همه تصمیم‌های بعدی روی همین عدد سوار است. در نسخه اصلی، خود کاربر q را اعلام می‌کرد («چقدر راحت یاد آمد؟») - اما خوداظهاری بین نوجوان‌ها نویز سنگین دارد. آزمونک q را **مصنوعی** می‌سازد از سه سیگنال عینی که از قبل داریم (تصمیم D10 و D14):

- **درستی** - پایه: پاسخ غلط، سقف q را زیر ۳ می‌برد؛ پاسخ درست، نقطه شروع بالا می‌دهد.

- **اعتماد (دکمه D14)** – «مطمئن بودم» ولی غلط $q = ۰$ پایین و ردپای بدفهمی؛ «حدس می‌زدم» ولی درست = سقف q در ۴ – چون حدس درست، حافظه واقعی را قوی‌تر نشان نکند.
- **تأخیر (آستانه‌های D10)** – پاسخ زیر ۱۵ ثانیه مشکوک به حدس رفلکسی است: سقف $q = ۴$. پاسخ بالای ۷۰ ثانیه یعنی سخت به جواب رسیده: -0.5 از q . هر سؤال می‌تواند با est_time_sec آستانه اختصاصی خودش را override کند (سؤال محاسباتی سنگین حق دارد زمان بیشتری بخواهد).

۶/۲ استخراج کامل: EF و بازگشت فواصل

قلب SM-2 دو عدد به‌ازای هر آیتم است: **آسانی نسبی EF** (شروع ۲/۵) و شماره تکرار n . آپدیت EF با کیفیت:

$$EF' = EF + (0.1 - (5-q) \cdot (0.08 + (5-q) \cdot 0.02))$$

قانون آپدیت آسانی نسبی — با کف ۱/۳ (6.1)

این فرمول را باید یک‌بار باز کنی تا بفهمی چقدر حساب‌شده است. بگذار $d = 5 - q$ (فاصله از کیفیت کامل):

$$EF' = EF + 0.1 - d \cdot 0.08 - d^2 \cdot 0.02$$

شکل بازشده — جریمه درجه دو برای کیفیت بد (6.2)

سه خاصیت از همین شکل درمی‌آید. در $q = 5$ هیچ تغییری نیست ($d = 0$) — پاسخ کامل، آسانی را دست نمی‌زند. در $q = 4$ هم تغییری نیست! ($0.1 - 1 \times 0.08 - 1 \times 0.02 = 0$) — و این دقیقاً همان جایی است که سقف «حدس زدم ولی درست» و «فوق‌سریع» در آزمونک، مهندسی‌شده است: حدس درست EF را نه بالا می‌برد و نه پایین — فقط n بالا می‌رود و فاصله بعدی کوتاه می‌ماند. از $q = 3$ به پایین جریمه به‌صورت درجه دوم رشد می‌کند ($-0.14 \rightarrow d = 2$ ؛ $-0.32 \rightarrow d = 3$) — یعنی سیستم روی «نمی‌دانم» بی‌رحم است اما روی «تقریباً بلد بودم» دست‌ودلباز. و کف ۱/۳ جلوی مرگ نرم‌شده آیتم را می‌گیرد: بدون کف، دنباله‌ای از پاسخ‌های بد، EF را تا صفر می‌برد و آیتم عملاً هر روز برمی‌گشت — تجربه‌ای که کاربر را از سیستم فرار می‌دهد.

فاصله مرور بعدی، بازگشتی ساده است:

$$I(1) = 1 ; I(2) = 6 ; I(n) = \lceil I(n-1) \times EF \rceil$$

دست‌نخورده از مقاله اصلی $I(2)=6$ فاصله‌ها به روز — سقف (6.3)

پاسخ‌های موفق، فاصله را هر بار در EF ضرب می‌کنند — رشد نمایی. پاسخ غلط، دنباله را به $n = 1$ برمی‌گرداند (نه صفر!) — چون آشنایی قبلی هنوز ارزش دارد و EF هم حفظ می‌شود؛ این نکته ریز، حس «سقوط از همه‌جا» را از کاربر می‌گیرد.

دو سرنوشت از یک مفهوم

مسیر الف – سه پاسخ درست با اعتماد بالا و زمان عادی: هر سه بار $q = 5$ ؛ پس EF همان $2/5$ می‌ماند. فاصله‌ها: $I(1) = 1$ روز $\leftarrow I(2) = 6$ روز $\leftarrow I(3) = \lceil 6 \times 2.5 \rceil = 15$ روز. بعد از سه هفته، این مفهوم فقط سه بار دیده شده و بعدی ۱۵ روز بعد است – این «شتاب‌گیری حافظه» است.

مسیر ب – همان ولی یکی‌اش حدس درست (q سقف ۴) و یکی غلط: $q = 5 \rightarrow 4 \rightarrow 0$. اولی EF را ثابت نگه می‌دارد (2.5) ولی برخورد بعدی غلط است: $EF' = 2.5 + 0.1 - 5 \times 0.08 = 2.5 - 0.7 = 1.8$ و $25 \times 0.02 = 2.5 - 0.7 = 1.8$ و شمارنده به $n = 1$ برمی‌گردد. فاصله بعدی: یک روز. سیستم دقیقاً همان‌جا و همان‌قدر محکم رفتار می‌کند که باید.

۶/۴ خانواده رقیب – از جعبه لایتنر تا FSRs

لایتنر (۱۹۷۲) – ساده‌ترین سیستم فاصله‌دار: پنج جعبه؛ پاسخ درست کارت را یک جعبه می‌برد بالا، پاسخ غلط به جعبه اول برمی‌گرداند؛ فاصله مرور هر جعبه ضربی از جعبه اول (مثلاً جعبه $n =$ هر n روز). همه چیز فیزیکی و قابل توضیح در ده ثانیه است؛ ضعف بزرگش، یکسان‌بودن ضریب برای همه کارت‌هاست – کارت سخت و کارت آسان در نهایت با همان سرعت جلو می‌روند. آزمونک با SM-2 همین ضعف را می‌بندد: EF هر آیتم شخصی می‌شود.

Pimsleur (۱۹۶۷) – پدر تجاری این حوزه برای یادگیری زبان: دنباله ثابت expanding rehearsal (۵ ثانیه، ۲۵ ثانیه، ۲ دقیقه، ۱۰ دقیقه، ۱ ساعت، ۵ ساعت، ۱ روز، ۵ روز، ۲۵ روز...). جالب است که Pimsleur از نظر زمانی نزدیک به SM-2 اولیه است (نسبت ~ 5 در گام‌های اول)، ولی چون برنامه ثابت است، هیچ شخصی‌سازی ندارد – نه EF، نه q ، نه بازگشت هوشمند پس از خطا. مقایسه این دو در دفاع، تمایز «برنامه‌بندی» از «تطبیق» را عالی نشان می‌دهد.

FSRS (Free Spaced Repetition Scheduler) – (یر، ۲۰۲۴) – نسل جدید و علمی‌ترین رقیب؛ جایگزین رسمی SM-2 در Anki از نسخه ۲۳/۱۰. مدل سه‌متغیره DSR دارد: سختی D ، ثبات S (چند روز تا افت به ریسک ۱۰٪) و بازیابی‌پذیری R که با قدرت لحظه‌ای حافظه است:

$$R(t, S) = (1 + \text{FACTOR} \cdot t/S)^{\text{DECAY}} ; \text{FACTOR} = 19/81 ; \text{DECAY} = -0.5$$

$R = 0.9$ دقیقاً $t = S$ در FSRs منحنی بازیابی (6.4)

زیرکشی (6.4): با ثابت‌های بالا، $R = (1 + t/9S)^{-0.5}$ و در $t = S$ می‌شود $(1 + 19/81)^{-0.5} = (100/81)^{-0.5}$ یعنی «ثبات» عمداً طوری تعریف شده که افت حافظه به ۹۰٪ باشد. FSRs فاصله بعدی را طوری می‌سازد که R به هدف دلخواه (مثلاً ۹۰٪) برسد، و ۱۷ تا ۲۱ وزن مدل را با بهینه‌سازی روی داده خود هر کاربر تنظیم می‌کند. نتیجه: دقت بالاتر و فاصله‌های بلندتر (تکرار کمتر برای همان ماندگاری). هزینه‌اش: نیاز به

داده انباشته برای آموزش وزن‌ها و پیچیدگی محاسبه‌ای چند سطح بالاتر از SM-2. برای آزمونک، SM-2 نقطه شروع درست است – ساده، بدون دیتا کار می‌کند، قابل توضیح – و در سند، ارتقا به مدل‌های $R(t,S)$ به‌عنوان فاز بعد ثبت نشده اما راهش باز است چون q آزمونک همان سیگنال ورودی را می‌سازد.

Half-Life Regression (دوئولینگو – ستلز و میدر، ۲۰۱۶) – رویکرد داده‌محور مطلق: نیم‌عمر حافظه هر آیت‌م را رگرسیون خطی روی ویژگی‌ها می‌کشد ($p = 2^{-\Delta/h}$ و $\log h = w \cdot x$ ؛ ویژگی‌هایی مثل تاریخچه درستی، فاصله، اعتماد). زیبایی‌اش این است که همه‌چیز از داده می‌آید و حتی ویژگی‌های متنی (چند حرفی بودن کلمه و...) را می‌بلعد؛ دوایرش: بدون داده انبوه از میلیون‌ها کاربر، و بی‌توجیه‌پذیری تک‌کاربر. دوئولینگو با صدها میلیون پاسخ این را جواب داد؛ ما با ۱۵۰ سؤال و ۴۰ آزمودنی – نه.

معیار	SM-2 (انتخاب ما)	لایتنر	Pimsleur	FSRS	HLR
شخصی‌سازی	EF و n به‌ازای هر آیت‌م	ندارد	ندارد	کامل (D,S,R)	کامل (h) از ویژگی‌ها
داده لازم برای شروع	صفر	صفر	صفر	آموزش وزن‌ها روی داده	داده انبوه
پیچیدگی پیاده‌سازی	کم – چند خط	بسیار کم	کم	بالا	بالا + زیرساخت ML
توضیح‌پذیری	بالا – EF قابل بازگویی	بسیار بالا	بالا	متوسط	ضعیف
مدل‌سازی حدس و تأخیر	بله – از (D10/D14) q	خیر	خیر	غیرمستقیم	به‌عنوان ویژگی
مناسب بانک ۱۵۰ سؤال و ۴۰ آزمودنی	بله	بله	بله	نه هنوز	نه

جدول ۱-۶ – مقایسه خانواده مرور فاصله‌دار. SM-2 تنها گزینه‌ای است که شخصی‌سازی واقعی را با «صفر داده اولیه» می‌دهد.

EF_floor

۱/۳

پایین‌تر بردنش = اجازه مرگ آیتم‌های مشکل‌دار (تجربه بد کاربر)؛ بالاتر = آیتم‌های ضعیف سریع‌تر نجات پیدا می‌کنند ولی فاصله‌ها غیرواقعی می‌شوند.

t_fast / t_slow

۱۵ ثانیه / ۷۰ ثانیه

تصمیم D10 پویا. بعد از MVP: دسته‌بندی سؤال‌ها و آستانه اختصاصی هر دسته با کالیبراسیون داده واقعی (سؤال محاسباتی حق زمان بیشتری دارد).

inject_review

سؤال مرور داخل جلسه عادی

نوآوری ۳ سند. اگر نرخ تکمیل صف مرور (هدف $\leq 60\%$) پایین آمد، اول اولویت تزریق را بالا ببر، نه نمایش بنر.

q mapping

سقف ۴ برای حدس/فوق‌سریع؛ -۵/۰ تأخیر

سخت‌گیرتر کردن این نگاشت = مرورهای سخت‌تر و فاصله‌های کوتاه‌تر. نگه‌دار این دستگیره، داده توزیع اعتماد (D14) است.

فصل ۷

پیش‌بینی نمره

دانش‌آموز به دانش‌آموز یک قول محتاطانه می‌دهد: «اگر روندت همین‌طور بماند، در آزمون جمع‌بندی حدوداً چقدر می‌گیری؟». این نوآوری ۵ سند است و دو قطعه دارد: اول باید «نمره جلسه» را طوری تعریف کنیم که منصفانه باشد (ورودی پیش‌بینی)، بعد روی دنباله نمره‌ها یک رگرسیون وزن‌دار بکشیم که هم عدد بدهد هم میزان اعتمادش را.

۷/۱ نمره جلسه وزن‌دار با سختی (D13)

ساده‌ترین تعریف، «تعداد درست ÷ تعداد سؤال» است — و بدترین. جلسه‌ای با ده سؤال آسان و جلسه‌ای با ده سؤال ناحیه سخت، هر دو می‌توانند «۸۰٪» بگیرند در حالی که توان واقعی‌شان فرق زمین تا آسمان دارد؛ رگرسیونی که روی این اعداد سوار شود، به دانش‌آموزی که سؤال‌های سخت‌تر می‌گیرد (چون توانش بالاست!) ظلم می‌کند و روندش را مصنوعاً صاف یا نزولی نشان می‌دهد. راه‌حل مصوب: وزن هر سؤال تابعی صعودی از سختی Elo آن، و نمره جلسه میانگین وزنی درستی‌ها:

$$\text{Score} = \frac{\sum (\text{weight}_i \times \text{correct}_i)}{\sum \text{weight}_i} \times 100 ; \text{weight} = f(R_q)$$

ساده و صعودی نگه داشته می‌شود (۱ / ۱٫۵ / ۲ برای سه رده) core در f نمره وزن‌دار — شکل (7.1)

مثال عددی سند – چرا وزن‌دهی منصفانه است

جلسه‌ای با ۶ سؤال آسان (وزن ۱)، ۳ متوسط (وزن ۱/۵) و ۱ سخت (وزن ۲): مجموع وزن‌ها $6 \times 1 + 3 \times 1.5 + 2 = 12.5$.

حالت الف: هفت درست از «۴ آسان + ۲ متوسط + ۱ سخت» ← نمره $(4 + 3 + 2)/12.5 \times 100 = 72$.

حالت ب: همان هفت درست، ولی همه از آسان‌ها ← نمره $7/12.5 \times 100 = 56$.

یک عدد درست‌پاسخ یکسان (۷) دو نمره کاملاً متفاوت – و دقیقاً منصفانه، چون سختی واقعاً پاسخ‌شده دیده می‌شود. این نمره، هم ورودی رگرسیون است هم خط اول نمودار روند (D15).

۷/۲ رگرسیون وزن‌دار نمایی – $\lambda = 0.85$

دنباله نمره‌های جلسات اخیر داریم و می‌خواهیم روند و ادامه آن را تخمین بزنیم. چرا «رگرسیون خطی ساده» نه؟ چون جلسه مهر ماه و جلسه امروز از نظر پیش‌بینی وضعیت **امروز** یک‌ارزش‌اند – در حالی که یادگیری، فراموشی و بلوغ، همه در «الان» جمع شده‌اند. آزمونک از میانگین‌گیری وزنی نمایی استفاده می‌کند: وزن جلسه‌ای که t جلسه قبل بوده $w = \lambda^t$ با $\lambda = 0.85$:

$$t_i = (\text{age of session } i) ; w_i = \lambda^{t_i} ; \text{ minimize } \sum w_i \cdot (y_i - a - b \cdot t_i)^2$$

با وزن نمایی WLS — کمینه‌سازی وزن‌دار خط برازش (7.2)

جواب این مسئله کمینه‌سازی، فرم بسته رگرسیون خطی است فقط با میانگین‌های وزنی: شیب b (روند) و عرض a از میانگین وزنی نقاط و کوواریانس وزنی به‌دست می‌آیند و پیش‌بینی جلسه بعد $\hat{y} = a + b \cdot t_{\text{next}}$ است. عدد $\lambda = 0.85$ از پیوست ب سند: جلسه قبلی وزن ۱، دو جلسه قبل 0.85 ، پنج جلسه قبل حدود 0.52 ، ده جلسه قبل 0.23 – یعنی ماه اخیر «خود دانش‌آموز» است، ماه قبل «پیشینه».

و خروجی فقط یک عدد نیست – **بازه اطمینان** دارد: انحراف معیار باقیمانده‌ها (σ) از ریشه میانگین وزنی مربعات خطا) محاسبه می‌شود و نمایش « 6 ± 78 » می‌شود. این بازه صادقانه است: دانش‌آموزی که نمره‌هایش پرش دارد، بازه پهن می‌بیند و خودش می‌فهمد تخمین، تخمین است. دو حکم محافظه‌کارانه هم داریم: زیر **۸ جلسه** اصلاً پیش‌بینی نمایش داده نمی‌شود (عدد گمراه‌کننده بدتر از بی‌عددی است – پیوست ب سند) و تابع نمره جلسه «به‌صورت قابل تعویض» طراحی می‌شود تا اگر بعداً تعریف بهتری پیدا شد (مثلاً نسخه پیوسته از R_s)، دیتای تاریخی با snapshot‌ها بازمحاسبه‌پذیر بماند.

رقیب‌ها: میانگین ساده آخرین n جلسه – بی‌سوگیری ندارد اما روند را کور است و واکنشش به تغییر یک‌هفته اخیر، مثل واکنشش به یک‌ماه پیش است؛ EMA ساده (میانگین متحرک نمایی بدون روند) – فقط «سطح فعلی» می‌دهد نه شیب، پس جمله «اگر همین‌طور ادامه بدهی» نمی‌شود ساخت؛ و پروجکشن مستقیم از Elo – «توان ۱۲۵۰ یعنی حدود x درصد» – که ساده است ولی جلسه‌های واقعی

(خستگی، اشتباه محاسباتی، انتخاب‌های ناحیه هدف) را مدل نمی‌کند و بازه اطمینان ندارد. رگرسیون وزن‌دار، بهترین تعادل «سادگی قابل دفاع / پاسخ به روند» است.

دستگیره‌های قابل تنظیم این فصل

min_sessions

۸ جلسه

کاهشش = پیش‌بینی زودتر اما ناپایدارتر؛ این عدد دفاع روانی هم دارد: «پیش از شناخت کافی، حرف نمی‌زنیم».

lambda

۰/۸۵

پایین‌تر (۰/۷۵) = حافظه کوتاه‌تر، واکنش سریع‌تر به «الان»؛ بالاتر (۰/۹۲) = صبورتر و کم‌نوسان‌تر. با نوسان واقعی نوجوان‌ها، ۰/۸۵ نقطه متعادل سند است.

weight_fn

۱ / ۱/۵ / ۲ (سه رده)

با نمونه‌های واقعی آزمایش میدانی می‌توانی پیوسته‌اش کنی (مثلاً $\exp((R_q - 1150)/400)$)؛ چون `params_snapshot` همه وزن‌های تاریخی را دارد، بازمحاسبه روند امن است.

فصل ۸

چارچوب اجرا و قواعد پشتیبان

الگوریتم‌ها موتور هستند، اما یک شاسی هم لازم است تا موتور روی داده واقعی منصفانه و تکرارپذیر کار کند. این فصل پنج قاعده شاسی‌ای است که همه فصل‌های قبلی به آن تکیه می‌کنند – الگوریتم نیستند، ولی اگر یکی‌شان برداشته شود، داده آزمایش علمی بی‌اعتبار می‌شود.

۱) تعریف جلسه و فاصله دو ساعت (D12). جلسه دقیقاً ۱۰ سؤال است (جلسه جایابی مستثنی است) و دو جلسه‌ای که به هم نزدیک‌تر از ۲ ساعت باشند، در قواعد «پیایی» و «دو شاهد» دو جلسه مستقل حساب نمی‌شوند. بدون این قاعده، دانش‌آموزی که پشت‌سرهم پنج جلسه می‌دهد هم استریک‌های نردبان را می‌شکند هم نمره‌های پیش‌بینی را؛ فاصله ۲ ساعته، «خنک‌شدن» را تضمین می‌کند که هر جلسه یک نمونه مستقل از وضعیت پایدار دانش‌آموز باشد – پیش‌نیاز هر آماری که روی جلسه‌ها حساب می‌کنیم.

۲) دکمه اعتماد (D14) – جریان سیگنال. بعد از انتخاب گزینه و قبل از نمایش بازخورد، دو دکمه بزرگ «مطمئن بودم / حدس می‌زدم». مکان انتخاب دقیق و مهم است: بعد از گزینه، چون دانش‌آموز حالا تصمیمش را گرفته و می‌تواند صادقانه بگوید چقدر رویش استوار بود؛ قبل از بازخورد، چون بعد از دیدن جواب، همه «مطمئن بودم» می‌زنند. این سیگنال وارد نمره جلسه نمی‌شود (تا نمره خالص بماند)؛ فقط به دو جا می‌رود: به `q` در SM-2 (فصل ۶) و به ردپای «مطمئن اما غلط» – قوی‌ترین الگوی پیش‌سرنخ بدفهمی برای روز فعال‌سازی موتور لایه ۲. هزینه‌اش فقط یک لمس در هر سؤال است که تست دود موبایل باید تحمل‌پذیری‌اش را تأیید کند.

۳) تراکنش اتمیک پاسخ (G5). یک پاسخ، پنج نوشتن همزمان می‌خواهد: رکورد attempts، آپدیت توان سراسری در ability_state، آپدیت ردیف دسته (سطح/استریک/شمارنده ضعیف – D16)، آپدیت review_schedule و snapshot پارامترها. همه در یک تراکنش دیتابیس: اگر هر بخش خطا دهد، کل تراکنش rollback می‌شود. چرا این قدر سخت‌گیری؟ چون داده آزمایش مقدس است – یک توان آپدیت‌شده ولی استریک نآپدیت‌شده، یعنی نردبان با حافظه مغشوش کار می‌کند و هیچ راه برگشتی هم نیست؛ در تست یکپارچگی گام ۲، خرابی عمدی وسط تراکنش شبیه‌سازی و rollback تأیید می‌شود.

۴) snapshot پارامترها (G7). هر رکورد attempt یک فیلد JSONB دارد که K_q و K_s لحظه‌ای، ناحیه هدف، نسخه الگوریتم و نسخه محتوا را ثبت می‌کند. چون همه پارامترهای این کتابچه «قابل و تو» و در طول عمر محصول ممکن است تغییر کنند، بدون snapshot هیچ تحلیلی نمی‌توانست بگوید «این افت سطح با قواعد قدیم بود یا جدید». با snapshot، تحلیل‌گر هر پاسخ را با پارامترهای لحظه پاسخ بازمحاسبه می‌کند – تکرارپذیری که داور جشنواره با آن «علمی بودن» را می‌سنجد.

۵) تصادفی‌سازی بلوکه‌شده (آزمایش میدانی). ۴۰ آزمودنی بر اساس نمره پیش‌آزمون مرتب می‌شوند، به بلوک‌های دوتایی هم‌تراز شکسته می‌شوند و در هر بلوک، قرعه (الگوریتم تخصیص در core و تست‌شده) یکی را گروه آزمایش (آزمونک کامل) و یکی را گروه کنترل (جلسه ثابت بدون تطبیق و بدون مرور) می‌کند. چرا بلوکه‌شده و نه قرعه ساده؟ چون با n کوچک (۲۰+۲۰)، قرعه ساده می‌تواند تصادفاً همه قوی‌ها را در یک گروه بریزد و اندازه اثر را بسوزاند؛ بلوکه‌سازی بر متغیر مهم (پیش‌آزمون) این ریسک را عملاً صفر می‌کند – و مقایسه پیش/پس‌آزمون با t مستقل و Cohen's d (ضمیمه د) معنادار می‌ماند.

جمع‌بندی فصل: جلسه = واحد اندازه‌گیری؛ فاصله ۲ ساعت = استقلال نمونه‌ها؛ دکمه اعتماد = سیگنال حافظه‌متا؛ تراکنش اتمیک = صحت داده؛ snapshot = تکرارپذیری؛ بلوکه‌سازی = اعتبار مقایسه علمی. این پنج تا هیچ‌کدام روی نمودار معماری نمی‌آیند، ولی بدونشان شش موتور، ماشین حافظه‌ناآگاه و بی‌اعتبار می‌شوند.

فصل ۹

جمع‌بندی تصمیم‌ها – چرا این‌ها؟

این فصل جدول دفاع نهایی است: برای هر لنگر الگوریتمی، گزینه برنده، گزینه‌های شاخص بازنده و دلیل مهندسی انتخاب. الگوی مشترک همه انتخاب‌ها یک چیز است: **بهترین ابزار برای مقیاس فعلی داده و تیم، نه بهترین ابزار دنیا.** هر جا که داده بزرگ شد، مسیر ارتقا باز است – و چون snapshot پارامترها داریم، ارتقا داده تاریخی را هم بی‌اعتبار نمی‌کند.

لنگر	انتخاب آزمونک	جایگزین‌های سنجیده‌شده	دلیل مهندسی انتخاب
سنجش توان	Elo دوطرفه با برنامه K	IRT 3PL، Glicko-2، TrueSkill	آپدیت لحظه‌ای با یک جمع / تفریق؛ صفر داده اولیه؛ توضیح‌پذیری کامل برای داور؛ حدس را با سیگنال انسانی (D14) جبران می‌کنیم نه پارامتر c
شروع سرد	جلسه جایابی ۴ سؤالی پراکنده، $K=32$	خوداظهاری؛ پیش‌فرض ثابت ۱۲۰۰؛ نمره پیش‌آزمون	خود موتور باید سطح را پیدا کند؛ پیش‌آزمون فقط برای بلوک‌سازی تا استقلال منابع حفظ شود
انتخاب سؤال	ناحیه هدف ۶۵-۷۵٪ + ارزش آموزشی + مواجهه ۲۰	Maximum Information (IRT)	ناحیه ۷۰٪ به جای ۵۰٪ = تجربه آموزشی‌تر؛ جای تعبیه ضعف دسته و اهمیت مبحث؛ جلوگیری از حفظ سؤال
تشخیص وضعیت مفهومی	نردبان سطح دسته‌ای (D16)	KST، DKT، BKT، مپ بدفهمی کامل	صفر بار محتوایی اضافه؛ سه قاعده کاملاً قابل توضیح؛ قواعد ثابت به داده کم‌حجم نیاز ندارند؛ مسیر فعال‌سازی BKT در آینده باز (فرمت حفظ شده)
مرور فاصله‌دار	SM-2 با q مصنوعی سه سیگنالی	لایت‌نر، FSRS، Pimsleur HLR	شخصی‌سازی واقعی (EF) با صفر دیتای اولیه؛ حدس/تأخیر داخل (D10/D14)؛ ساده و تست‌طلبی‌پذیر؛ ارتقا به مدل $R(t,S)$ در آینده ممکن
پیش‌بینی	رگرسیون WLS وزن نمایی $\lambda=0/85 +$ بازه اطمینان	میانگین ساده؛ EMA بدون روند؛ پروچکشن Elo	روند را مدل می‌کند نه فقط سطح؛ وزن نمایی = بهینگی با حافظه نوجوان؛ زیر ۸ جلسه سکوت – صداقت علمی
نمره جلسه	وزن‌دار با سختی (D13)	درصد خام؛ R_S نگاشت‌شده	منصفانه بین جلسه سخت و آسان؛ ورودی پایدار برای رگرسیون؛ مثال ۷۲/۵۶ قابل ارائه به داور

جدول ۹-۱ – نقشه انتخاب‌ها. هر تصمیم در سند برنامه (D1-D16) ثبت شده و قابل و تواتر است.

و در نهایت، نقشه دستگیره‌های کل سیستم در یک نگاه – این جدول پاسخ سؤال «اگر بخواهم یک چیز را تنظیم کنم، کجای این سیستم را بچرخانم؟» است. ترتیب ردیف‌ها از «امن‌ترین تغییر» تا «جراحی» است: همیشه از بالای جدول شروع کن، زیرا بالایی‌ها اثر موضعی دارند و پایینی‌ها رفتار کل سیستم را می‌سازند.

دستگیره	پیش‌فرض	جهت‌گیری اثر
مپ q (سقف حدس / فوق‌سریع، -۰/۵ تأخیر)	D10/D14	موضعی‌ترین دستگیره – فقط فاصله‌های مرور عوض می‌شوند
ناحیه هدف انتخاب	۰/۷۵-۰/۶۵	احساس چالش جلسه‌ها؛ حساسیت‌سنجی در آزمایش
m و n نردبان	۲ / ۲	سرعت و پایداری تغییر سطح هر دسته
λ رگرسیون پیش‌بینی	۰/۸۵	حافظه روند داشبورد
exposure_window	۲۰	تنوع سؤال در برابر مصرف بانک
وزن‌های ارزش آموزشی w1/w2/w3	پیش‌فرض مهندس	شخصیت هدایت‌گر سیستم (ضعف/اهمیت/recency)
برنامه K	۴→۸؛ ۱۶→۲۴→۳۲	سرعت همگرایی توان و سختی – تغییرش داده تاریخی را بازمقایسه‌ناپذیر می‌کند (snapshot جبران می‌کند)
کف EF و I(1), I(2)	۱/۳؛ ۱ و ۶	جراحی – مقیاس کل مرور جابه‌جا می‌شود

جدول ۹-۲ – نقشه تنظیم: از امن به جراحی. هر تغییر مهم، در سند برنامه ثبت تصمیم می‌شود (پیوست ب، قاعده ۶).

ضمیمه الف

جدول کامل پارامترها

همه ثابت‌های الگوریتمی در یک جدول – مقدار اولیه، منبع تصمیم و اثر تغییر. همه این‌ها در `params_snapshot` هر پاسخ ثبت می‌شوند و همه (به‌جز مقیاس Elo که زیرساخت اعداد است) «قابل وتو» هستند.

پارامتر	مقدار اولیه	منبع	اثر تغییر
K دانش‌آموز (شروع)	۳۲	سند – پیوست ب	بالا = همگرایی سریع‌تر و نوسان بیشتر
K دانش‌آموز (مرحله ۲/۳)	۲۴ پس از ۵۰؛ ۱۶ پس از ۱۵۰	سند	آستانه‌ها = زمان ورود به فاز پایداری

پارامتر	مقدار اولیه	منبع	اثر تغییر
K سؤال	۸ ← ۴ پس از ۴۰ پاسخ	سند	بالا = جذب سریع تر رأی جامعه بر سختی
ناحیه هدف انتخاب	۰/۶۵ - ۰/۷۵ (مرجع ۰/۷۰)	سند	چالش/اعتماد جلسه ها؛ حساسیت سنجی در آزمایش
پنجره مواجهه	۲۰ پاسخ	G8	تنوع در برابر مصرف بانک سؤال
طول جایابی / K جایابی	۴ سؤال (۳-۵) / ۳۲ ثابت	D1	دقت تخمین اولیه در برابر تجربه ورود
سؤال تکمیلی جایابی	۲ (روی پراکندگی بالا)	D1	سقف حفاظتی حلقه شناخت
n افت نردبان / m ارتقا	۲ / ۲	D16	پایداری سطح ها؛ n=1 پرنوسان، m=1 شکننده
سقف/کف نردبان	سطح ۱ تا ۳ ← ۱۱۵۰/۱۲۵۰/۱۳۵۰	سند + D16	نوار سختی انتخاب سؤال هر دسته
حد ارتقا در جلسه	یک بار برای هر دسته	D16	جلوی پرش کاذب در جلسه شانس آور
هشدار مطالعه	سطح ۱ + موفقیت >۳۰٪ در ۵ پاسخ	D16	حساسیت بنر - بالا = کری هشدار
هشدار نوسان دبیر	۳ ≤ تغییر در ۵ جلسه	گام ۴/۴	پرشدگی صف بازبینی دبیر
t_fast / t_slow	۱۵ ثانیه / ۷۰ ثانیه	D10	کیفیت q؛ کالیراسیون per-category پس از MVP
کف EF	۱/۳	سند	نجات پذیری آیتم های ضعیف
I(1), I(2)	۱ روز، ۶ روز	SM-2 اصلی	مقیاس کل زنجیره مرور
فاصله حداقلی جلسات	۱۲۰ دقیقه	D12	استقلال نمونه های جلسه
طول جلسه	۱۰ سؤال	سند + D12	خستگی/تعهد - جایابی مستثنی
وزن نمره جلسه (سه رده)	۱ / ۱/۵ / ۲	D13	عدالت نمره در برابر سختی؛ قابل پیوسته شدن
λ روند پیش بینی	۰/۸۵	سند - پیوست ب	حافظه روند داشبورد

پارامتر	مقدار اولیه	منبع	اثر تغییر
حداقل جلسه برای پیش‌بینی	۸	سند	زمان سکوت پیش‌بینی
مقیاس Elo	۴۰۰ (توان ۱۱۰۰- ۱۴۰۰)	سند	جراحی – همه ثابت‌ها جابه‌جا می‌شوند

جدول الف-۱ – پارامترهای الگوریتمی. نسخه‌دار با نسخه الگوریتم در snapshot.

ضمیمه ب

ده پرسش محتمل داوری و پاسخ کوتاه

(۱) چرا Elo و نه IRT که استاندارد علمی است؟ IRT برای بازآورد دوره‌ای روی داده انبوه طراحی شده؛ ما به آپدیت لحظه‌ای در هر پاسخ، کارکرد با بانک کوچک و توضیح‌پذیری دوعده‌ای نیاز داریم. ساختاری، Elo زیرمجموعه IRT دوپارامتری با شیب ثابت است – یعنی مسیر ارتقا به IRT هم بعداً باز است.

(۲) شانس حدس در چهارگزینه‌ای را چطور مدل می‌کنید؟ مدل‌سازی آماری c را با سیگنال انسانی جبران می‌کنیم: دکمه اعتماد (D14). «حدس می‌زدم» درست سقف q در ۴ می‌گیرد و در Elo، حدس درست فراوان، توان را با نرخ بالاتر انتظار طبیعی تعدیل می‌کند.

(۳) چرا ناحیه هدف ۷۰٪ است نه ۵۰٪؟ ۵۰٪ نقطه بهینه «اطلاعات آماری» است، اما تجربه آموزشی پرفشار و کم‌بازخوردی می‌سازد؛ ۶۵-۷۵ تعادل انگیزه/چالش سند است و در آزمایش میدانی حساسیت‌سنجی می‌شود – یعنی پاسخ ما داده‌محور است، لجبازانه نیست.

(۴) اگر دانش‌آموز غلط‌هایش تصادفی باشد چه؟ استریک «پیایی» به‌طور طراحی برای همین است: الگوی پراکنده، سطح را نمی‌اندازد؛ الگوی متمرکز می‌اندازد. و هشدار نوسان، موارد عجیب را به دبیر انسانی می‌برد – الگوریتم جای انسان را نمی‌گیرد.

(۵) چرا SM-2 قدیمی و نه FSRS که دقیق‌تر است؟ FSRS برای آموزش وزن‌ها به داده انبوه نیاز دارد و خروجی‌اش برای نوجوان قابل بازگویی نیست؛ SM-2 صفر داده اولیه می‌خواهد، EF آن یک جمله توضیح دارد و q ما سه سیگنال عینی دارد. دقیق‌تر بودن FSRS در مقیاس ما هنوز پا روی پاکت داده نمی‌کند.

(۶) مرور چطور اپ رهاشده نمی‌شود؟ چون صف مرور جداگانه نیست – تزریق در جلسه عادی است (نوآوری ۳ سند) و نرخ تکمیلش (هدف $\leq 60\%$) شاخص فرعی آزمایش است.

(۷) اعتماد خوداظهاری نوجوان قابل اعتماد است؟ سیگنال‌های قضاوت‌های نسبی سنجیده می‌شود نه مطلق: جفت «مطمئن اما غلط» ارزشمند است حتی اگر مطلق‌ها نویز داشته باشند؛ و چون یکنواخت روی همه سؤال‌ها جمع می‌شود (ب)، سوگیری گزارش بین دانش‌آموزان مقایسه‌پذیر می‌ماند.

(۸) پیش‌بینی چرا زیر ۸ جلسه نمی‌دهید؟ چون رگرسیون روی ۳ جلسه، عدد قشنگ دروغ تولید می‌کند – ریسک «پیام اشتباه به نوجوان» از منفعت محدودش بیشتر است؛ سکوت، تصمیم طراحی است نه نقص.

۹) داده آزمایش چطور از خرابی محفوظ است؟ تراکنش اتمیک (G5) + snapshot پارامترها (G7) + هیچ پاک‌سازی دستی (پیوست ب سند). هر پاسخ، با پارامترهای لحظه‌اش بازتولیدپذیر است.

۱۰) اثرگذاری را چطور اثبات می‌کنید؟ پیش‌آزمون/پس‌آزمون موازی، بلوک‌سازی بر پیش‌آزمون، گروه کنترل ۲۰ نفره، t مستقل و Cohen's d – اسکریپت تحلیل متن‌باز در مخزن تا داور خودش اجرا کند.

ضمیمه ج

واژه‌نامه اصطلاحات

اصطلاح (انگلیسی)	معادل و توضیح فارسی
Ability (θ / R_s)	توان – تخمین عددی سطح دانش‌آموز روی مقیاس Elo
Difficulty (b / R_q)	سختی – ریتینگ سؤال؛ با رأی جامعه آزمونک به مقدار واقعی میل می‌کند
Expected score (E)	انتظار – شانس درستی پیش از پاسخ، از فرمول لجستیک
K-factor	ضریب گام – حداکثر سرعت حرکت ریتینگ در هر پاسخ
Cold start	شروع سرد – نبود داده برای تخمین اولیه؛ راه‌حل ما: جایابی
Item Response Theory (IRT)	نظریه پاسخ به سؤال – خانواده مدل‌های 1/2/3PL با تمایز و حدس
Rating deviation (RD)	انحراف ریتینگ – عدم قطعیت تخمین در Glicko-2
Knowledge tracing (BKT/DKT)	ردیابی دانش – برآورد احتمال تسلط مفهومی (بیزی / عصبی)
Knowledge Space Theory (KST)	نظریه فضای دانش – حالت‌های مجاز دانش و آزمون تطبیقی صوری
Spaced repetition	مرور فاصله‌دار – تکرار در لحظه‌های قبل از فراموشی
Easiness factor (EF)	آسانی نسبی – ضریب رشد فاصله مرور هر آیتم در SM-2
Interval $I(n)$	فاصله مرور n -ام – به روز؛ رشد نمایی با EF
Retrievability (R)	بازیابی‌پذیری – احتمال یادآوری الان در مدل‌های $R(t,S)$ مثل FSRS
Stability (S)	ثبات – چند روز طول می‌کشد تا R به ۹۰٪ برسد
Half-life (h)	نیم‌عمر – زمان تا افت احتمال یادآوری به ۵۰٪ (HLLR دوئولینگو)
Exposure window	پنجره مواجهه – ممنوعیت تکرار سؤال دیده‌شده در ۲۰ پاسخ اخیر

اصطلاح (انگلیسی)	معادل و توضیح فارسی
Target zone	ناحیه هدف – بازه شانس موفقیت مجاز برای سؤال بعدی (۶۵–۷۵٪)
Weighted session score	نمره جلسه وزن‌دار – میانگین درستی‌ها با وزن سختی (D13)
Exponentially weighted regression	رگرسیون وزن‌نمایی – وزن جلسات با λ^t ، این‌جا $\lambda=0.85$
Confidence interval	بازه اطمینان – $\pm$ انحراف معیار باقیمانده دور پیش‌بینی
Atomic transaction	تراکنش اتمیک – همه‌چیز یا هیچ‌چیز در نوشتن یک پاسخ
Params snapshot	گزارش پارامترها – JSONB هر پاسخ برای بازمحاسبه تکرارپذیر
Blocked randomization	تصادفی‌سازی بلوکه‌شده – تقسیم هم‌تراز بر پیش‌آزمون
Cohen's d	اندازه اثر – اختلاف میانگین‌ها به مقیاس انحراف معیار مشترک

ضمیمه د

مراجع علمی

- Elo, A. E. (1978). *The Rating of Chessplayers, Past and Present*. Arco – مرجع اصلی موتور Elo و مقیاس ۴۰۰.
- Bradley, R. A., & Terry, M. E. (1952). Rank analysis of incomplete block designs. *Biometrika*, 39(3/4), 324–345 – مدل زیربنایی فرمول انتظار.
- Woźniak, P. A. (1990). *Optimization of Repetitions in Spaced Repetition Practice*. University of Technology in Poznan – مقاله اصلی SM-2، فرمول EF و $I(n)$.
- Ebbinghaus, H. (1885). *Über das Gedächtnis*. – منحنی فراموشی و توجیه تزریق مرور.
- Leitner, S. (1972). *So lernt man lernen*. – سیستم جعبه‌ها.
- Pimsleur, P. (1967). A memory schedule. *Modern Language Journal*, 51(2), 73–75 – دنباله ثابت expanding rehearsal.
- Rasch, G. (1960). *Probabilistic Models for Some Intelligence and Attainment Tests*. – ریشه 1PL.
- Lord, F. M. (1980). *Applications of Item Response Theory to Practical Testing Problems*. Routledge. – IRT دو و سه‌پارامتری.
- Glickman, M. E. (1999). Parameter estimation in large dynamic paired comparison experiments. *Applied Statistics*, 48(3), 377–394. – Glicko و (2012) مثال عددی Glicko-2.

- – Herbrich, R., Minka, T., & Graepel, T. (2006). TrueSkill: A Bayesian skill rating system. *NIPS 19*
ریتینگ گاوسی و گراف فاکتور.
- Corbett, A. T., & Anderson, J. R. (1994). Knowledge tracing: Modeling the acquisition of procedural
knowledge. *UMUAI, 4*, 253–278. – چهار پارامتر BKT.
- LSTM و Piech, C., et al. (2015). Deep knowledge tracing. *NeurIPS 28*. – DKT
- Doignon, J.-P., & Falmagne, J.-C. (1985). Spaces for the assessment of knowledge. *International
Journal of Man-Machine Studies, 23*, 175–196. – KST
- Settles, B., & Meeder, B. (2016). A trainable spaced repetition model for language learning (HLR).
ACL 2016, 1848–1858. – نیم عمر رگرسیون دوئولینگو.
- – Ye, J. (2024). *FSRS: A modern spaced repetition algorithm*. open-spaced-repetition (srs-maker)
مدل DSR و ثابت‌های DECAY/FACTOR.

الگوریتم‌ها مالکیت می‌خواهند، نه حفظ کردن.

این کتابچه طوری نوشته شده که هر فصل را بتوانی مستقل باز کنی: فرمول، مثال عددی، رقیب، و دستگیره‌های تنظیم. هر تغییر مهمی که در پارامترها دادی، یک خط در جدول تصمیم‌های سند برنامه (D-جدید) ثبت می‌شود و snapshot پارامترها تضمین می‌کند داده‌های قبلی بازمقایسه‌پذیر بمانند. سند زنده است – مثل خود پروژه.